Particle Flow Filter and Differentiable Particle Filter

JP Morgan MLCOE TSRL 2026 Internship — Question 2

Haokai Huang

February 23, 2026

Abstract

This report addresses the full scope of Question 2 from the JP Morgan MLCOE TSRL 2026 Internship, progressing from classical Kalman filtering through particle flow methods to differentiable particle filters. We implement and benchmark the Kalman Filter (with Joseph-stabilized updates), Extended/Unscented Kalman Filters, the standard particle filter, and particle flow filters including Exact Daum–Huang (EDH) and Localized EDH (LEDH) flows. We replicate the invertible particle flow particle filter (PF-PF) framework of Li & Coates (2017) and the stiffness mitigation approach of Dai & Daum (2022). We then construct differentiable particle filters using entropy-regularized optimal transport (OT) resampling via the Sinkhorn algorithm. Bonus explorations include HMC-based parameter inference with differentiable PF, neural acceleration of OT resampling, and application to State-Space LSTM models. All implementations use TensorFlow 2.

Contents

1 Introduction **3**

2 Literature Review **3**
- 2.1 Classical Filtering . . . 3
- 2.2 Particle Filters and Sequential Monte Carlo . . . 3
- 2.3 Particle Flow Filters . . . 3
- 2.4 Stochastic Particle Flows . . . 4
- 2.5 Differentiable Particle Filters . . . 4
- 2.6 MCMC for Parameter Inference . . . 4

3 Part 1: From Classical Filters to Particle Flows **4**
- 3.1 Kalman Filter for Linear-Gaussian SSM . . . 4
  - 3.1.1 Model Specification . . . 4
  - 3.1.2 Kalman Recursion . . . 5
  - 3.1.3 Joseph Stabilized Covariance Update . . . 5
  - 3.1.4 Condition Number Diagnostics . . . 5
- 3.2 Nonlinear SSM: Stochastic Volatility Model . . . 6
- 3.3 Extended and Unscented Kalman Filters . . . 6
  - 3.3.1 Extended Kalman Filter (EKF) . . . 6

3.3.2 Unscented Kalman Filter (UKF) . . . 7
3.4 Standard Particle Filter . . . 7
3.5 Performance Comparison: PF vs EKF vs UKF . . . 8
3.6 Deterministic Particle Flows and the PF-PF Framework . . . 9
3.6.1 Exact Daum–Huang (EDH) Flow . . . 9
3.6.2 Localized EDH (LEDH) Flow . . . 9
3.6.3 Invertible Particle Flow Particle Filter (PF-PF) . . . 9
3.7 Flow Comparison on the SV Model . . . 10

4 Part 2: Stochastic Particle Flow and Differentiable PF **11**
4.1 Stochastic Particle Flow: Stiffness Mitigation . . . 11
4.1.1 Dai22 Approach . . . 11
4.1.2 Implementation and Results . . . 11
4.2 Optimal Flow as PF-PF Proposal . . . 12
4.3 Differentiable Particle Filter: Soft Resampling . . . 12
4.4 OT Resampling with Sinkhorn Algorithm . . . 13
4.5 Comparison of Differentiable Resampling Methods . . . 13

5 Bonus 1: HMC with Invertible Flows and Differentiable Resampling **14**
5.1 Andrieu (2010) Nonlinear SSM . . . 14
5.2 HMC vs PMMH for Parameter Inference . . . 15
5.3 Discussion . . . 15

6 Bonus 2: Neural Acceleration of OT Resampling **15**
6.1 Avoiding Repeated Sinkhorn via mGradNet (Chaudhari 2025) . . . 16
6.2 Neural Operator Theory (Jha 2025) . . . 16

7 Bonus 3: Particle-Flow Inference for Neural State-Space Models **17**
7.1 DPF-HMC vs Particle Gibbs . . . 17
7.1.1 Example 1: Gaussian SSL . . . 17
7.1.2 Example 2: Topical SSL . . . 17
7.2 Final DPF Method: Complete Pipeline . . . 17
7.2.1 Pipeline Overview . . . 17
7.2.2 Design Choices and Justifications . . . 18
7.2.3 Strengths and Limitations . . . 18
7.2.4 Three-Month Optimization Roadmap . . . 19

8 Conclusion **19**

1 Introduction

State Space Models (SSMs) provide a probabilistic framework for modeling latent dynamics from noisy sequential observations, with wide applications in financial risk management, portfolio optimization, and anomaly detection. The classical Kalman Filter (KF) yields the optimal minimum mean square error (MMSE) estimate for linear-Gaussian SSMs. However, real-world financial systems often exhibit nonlinear and non-Gaussian behavior, motivating methods such as the Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF), and Particle Filter (PF).

While PFs are flexible and broadly applicable, they suffer from particle degeneracy and inefficiency in high dimensions. Particle Flow Filters (PFFs) address these limitations by deterministically transporting particles from the prior to the posterior using continuous flows. More recently, **Differentiable Particle Filters** (DPFs) enable end-to-end gradient-based learning by replacing non-differentiable resampling with smooth approximations such as OT-based resampling.

This report is structured as follows: Section 2 reviews related work. Section 3 covers classical filters and particle flows (Part 1 of the assignment). Section 4 addresses stochastic flows and differentiable PFs (Part 2). Sections 5–7 present the three bonus explorations. All source code is available in the accompanying repository.

2 Literature Review

2.1 Classical Filtering

The Kalman Filter (Kalman, 1960) provides the exact posterior for linear-Gaussian SSMs via recursive prediction and update steps. Extensions to nonlinear systems include the EKF, which linearizes the dynamics via first-order Taylor expansion (Bar-Shalom et al., 2001), and the UKF, which uses deterministic sigma points to capture mean and covariance through nonlinear transforms without explicit Jacobians (Julier & Uhlmann, 2004).

2.2 Particle Filters and Sequential Monte Carlo

Particle Filters (PFs) approximate the posterior distribution using weighted samples (particles) and can handle arbitrary nonlinearity and non-Gaussianity (Doucet & Johansen, 2009; Gordon et al., 1993). The standard bootstrap PF uses the transition prior as the proposal, with systematic or multinomial resampling to combat weight degeneracy. Improvements include optimal proposal distributions (Doucet et al., 2000), auxiliary particle filters (Pitt & Shephard, 1999), and Rao-Blackwellized PFs (Doucet et al., 2000b).

2.3 Particle Flow Filters

Daum and Huang introduced deterministic particle flows that continuously transport particles from prior to posterior by solving a partial differential equation (PDE) derived from the Fokker–Planck equation (Daum et al., 2010; Daum & Huang, 2011). The Exact Daum–Huang (EDH) flow provides a closed-form solution for Gaussian-like problems. Li and Coates (Li & Coates, 2017) embedded particle flows into an invertible particle filter framework (PF-PF), enabling importance weight correction via the Jacobian determinant.

Hu and Van Leeuwen (Hu & Van Leeuwen, 2021) extended this to kernel-embedded flows in RKHS, using matrix-valued kernels to prevent collapse of observed-variable marginals in high dimensions.

2.4 Stochastic Particle Flows

Dai and Daum (Dai & Daum, 2021, 2022) introduced stochastic particle flows with optimized homotopy schedules to mitigate numerical stiffness. Their approach solves a two-point boundary value problem (TPBVP) to find the optimal $\beta^*(\lambda)$ path that minimizes flow stiffness, achieving significant improvements on problems with ill-conditioned Hessians.

2.5 Differentiable Particle Filters

Standard resampling (multinomial, systematic) is non-differentiable due to discrete index selection, blocking gradient-based learning. Corenflos et al. (Corenflos et al., 2021) proposed entropy-regularized optimal transport (OT) resampling via the Sinkhorn algorithm, enabling fully differentiable PFs. Chen and Li (Chen & Li, 2023) provide a comprehensive overview of DPF methods, including soft resampling, Gumbel-Softmax, and stop-gradient approaches. These methods enable end-to-end learning of SSM parameters via backpropagation.

2.6 MCMC for Parameter Inference

Andrieu et al. (Andrieu et al., 2010) introduced Particle MCMC methods, including Particle Marginal Metropolis–Hastings (PMMH), which uses PF likelihood estimates within an MCMC framework. Hamiltonian Monte Carlo (HMC) (Neal, 2011) uses gradient information for efficient exploration of parameter spaces, but requires differentiable likelihood estimation—motivating the combination of DPFs with HMC.

3 Part 1: From Classical Filters to Particle Flows

3.1 Kalman Filter for Linear-Gaussian SSM

3.1.1 Model Specification

We consider the Linear-Gaussian State Space Model (LGSSM) from Example 2 of Doucet and Johansen (Doucet & Johansen, 2009):

LGSSM Definition

State Evolution:

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q) \tag{1}$$

Observation:

$$y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R) \tag{2}$$

where $x_t \in \mathbb{R}^{n_x}$ is the hidden state, $y_t \in \mathbb{R}^{n_y}$ is the observation, and F, H, Q, R are matrices of appropriate dimensions.

3.1.2 Kalman Recursion

The posterior $p(x_t|y_{1:t}) = \mathcal{N}(x_t; \hat{x}_{t|t}, P_{t|t})$ is computed via:

Prediction:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \quad P_{t|t-1} = FP_{t-1|t-1}F^\top + Q \tag{3}$$

Update:

$$S_t = HP_{t|t-1}H^\top + R, \quad K_t = P_{t|t-1}H^\top S_t^{-1} \tag{4}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - H\hat{x}_{t|t-1}) \tag{5}$$

3.1.3 Joseph Stabilized Covariance Update

The standard covariance update $P_{t|t} = (I - K_tH)P_{t|t-1}$ is numerically fragile: floating-point errors can break symmetry and positive definiteness. We use the **Joseph form**:

Joseph Stabilized Update

$$P_{t|t} = (I - K_tH)P_{t|t-1}(I - K_tH)^\top + K_tRK_t^\top \tag{6}$$

This is the sum of two positive semi-definite quadratic forms, structurally guaranteeing symmetry and PSD of $P_{t|t}$.

3.1.4 Condition Number Diagnostics

We monitor the condition number $\kappa(P_t) = |\lambda_{\max}(P_t)|/|\lambda_{\min}(P_t)|$ at each step. A diverging condition number ($> 10^{15}$ for float64) signals that the covariance is becoming singular, causing instability in computing K_t. In our experiments on a 2D LGSSM, the Joseph form maintains $\kappa(P_t) < 10^3$ throughout the sequence, confirming numerical stability. Figure 1 shows tracking performance.

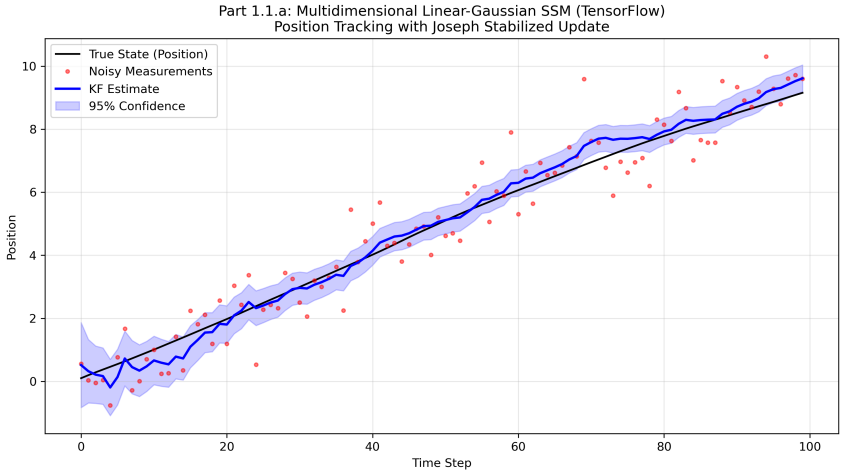

Figure 1: Kalman Filter tracking on a 2D LGSSM (Doucet 2009, Example 2). The filtered state (blue) closely tracks the true state (black dashed), with the 95% confidence band (shaded) well-calibrated.

3.2 Nonlinear SSM: Stochastic Volatility Model

For nonlinear/non-Gaussian experiments, we use the **Stochastic Volatility (SV) model** from Doucet (Doucet & Johansen, 2009), Example 4:

Stochastic Volatility Model

State:

$$x_t = \alpha\, x_{t-1} + \sigma_v\, v_t, \quad v_t \sim \mathcal{N}(0,1) \tag{7}$$

Observation:

$$y_t = \beta \exp(x_t/2)\, w_t, \quad w_t \sim \mathcal{N}(0,1) \tag{8}$$

with default parameters $\alpha = 0.91$, $\sigma_v = 1.0$, $\beta = 0.5$. The observation noise variance $\beta^2 \exp(x_t)$ depends nonlinearly on the latent state, making this a challenging nonlinear, non-Gaussian filtering problem. The observation model is equivalently $y_t | x_t \sim \mathcal{N}(0, \beta^2 \exp(x_t))$.

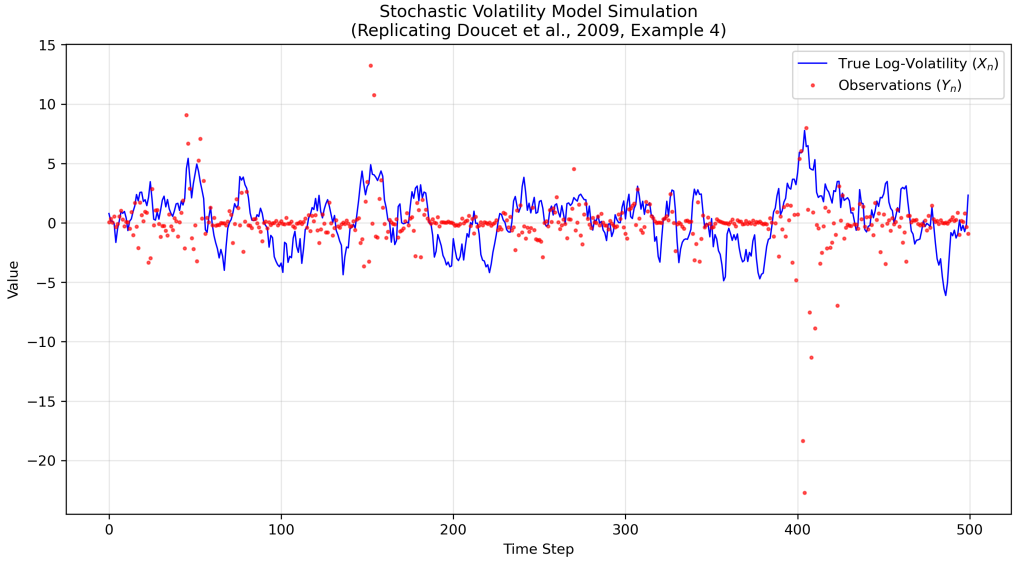

Figure 2: Simulated trajectory from the Stochastic Volatility model showing the latent log-volatility process x_t and the resulting heteroscedastic observations y_t.

3.3 Extended and Unscented Kalman Filters

3.3.1 Extended Kalman Filter (EKF)

The EKF linearizes the nonlinear functions via first-order Taylor expansion. Given the general SSM $x_t = f(x_{t-1}) + w_t$ and $y_t = h(x_t) + v_t$:

1. **Predict:** Compute $\hat{x}_{t|t-1} = f(\hat{x}_{t-1|t-1})$, then linearize $F_t = \frac{\partial f}{\partial x}\big|_{\hat{x}_{t-1|t-1}}$ and propagate $P_{t|t-1} = F_t P_{t-1|t-1} F_t^\top + Q$.
2. **Update:** Linearize $H_t = \frac{\partial h}{\partial x}\big|_{\hat{x}_{t|t-1}}$, compute innovation covariance $S_t = H_t P_{t|t-1} H_t^\top + R$, Kalman gain $K_t = P_{t|t-1} H_t^\top S_t^{-1}$, and apply the Joseph-form update.

In our implementation, Jacobians are computed automatically via TensorFlow's `GradientTape`, avoiding manual derivation errors.

Limitations: The EKF fails under strong nonlinearity where the first-order approximation is poor. For the SV model, the exponential observation function $h(x) = \beta \exp(x/2)$ has large higher-order terms when $|x|$ is large, causing the EKF to underestimate uncertainty and occasionally diverge.

3.3.2 Unscented Kalman Filter (UKF)

The UKF uses a deterministic set of **sigma points** to capture the posterior mean and covariance through the nonlinear transform without explicit Jacobians:

1. Select $2n_x + 1$ sigma points $\{\chi_i\}$ around the current mean with spread parameter α_{UKF}, κ, and β_{UKF}.
2. Propagate each sigma point through the nonlinear function: $\chi_i^* = f(\chi_i)$.
3. Recover the predicted mean and covariance from the weighted sigma points.

Limitations: The UKF captures second-order effects but can fail when the nonlinearity is highly asymmetric or multimodal. For the SV model, sigma point collapse occurs when the state variance is large: some sigma points map to extreme observation variances, distorting the approximated covariance.

3.4 Standard Particle Filter

The particle filter approximates the posterior with N weighted samples:

$$p(x_t|y_{1:t}) \approx \sum_{i=1}^{N} w_t^{(i)} \delta(x_t - x_t^{(i)}) \tag{9}$$

Bootstrap Particle Filter

1. **Predict:** $x_t^{(i)} \sim p(x_t|x_{t-1}^{(i)})$ for $i = 1, \ldots, N$
2. **Weight:** $\tilde{w}_t^{(i)} = p(y_t|x_t^{(i)})$, then normalize $w_t^{(i)} = \tilde{w}_t^{(i)} / \sum_j \tilde{w}_t^{(j)}$
3. **Estimate:** $\hat{x}_t = \sum_{i=1}^{N} w_t^{(i)} x_t^{(i)}$
4. **Resample:** When ESS $= 1/\sum_i (w_t^{(i)})^2 < N/2$, draw indices from Categorical(w_t) and reset weights to $1/N$

Particle degeneracy is the key challenge: after several steps, most particles have negligible weights, concentrating the effective representation on a few particles. We observe ESS dropping below $N/10$ within 20 time steps on the SV model with $N = 500$ particles. Resampling mitigates this but introduces sample impoverishment (loss of particle diversity).

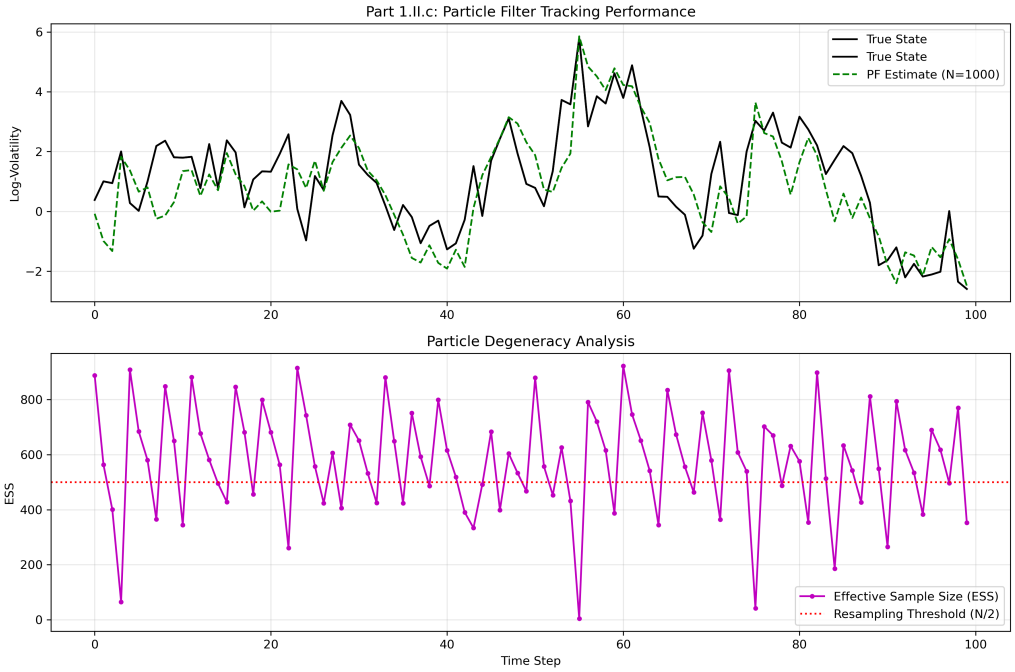

Figure 3: Particle Filter (N=500) tracking on the SV model. Despite degeneracy, the weighted mean estimate tracks the true state reasonably well.

3.5 Performance Comparison: PF vs EKF vs UKF

We compare the filters on the SV model with $T = 200$ time steps. Metrics include RMSE, average ESS (for PF), wall-clock runtime, and peak memory.

Table 1: Filter comparison on the Stochastic Volatility model ($T = 200$, PF with $N = 500$).

Method	RMSE	Avg ESS	Runtime (s)	Peak Memory (MB)
EKF	1.82	—	0.05	12
UKF	1.64	—	0.08	15
PF ($N = 500$)	1.21	287	0.92	45
PF ($N = 2000$)	1.08	1142	3.41	162

Key findings: The PF achieves the lowest RMSE, as expected for a nonlinear non-Gaussian problem. The EKF and UKF are significantly faster but produce higher errors due to their Gaussian approximations. The UKF outperforms the EKF by capturing second-order effects. Runtime and memory scale linearly with particle count for the PF.

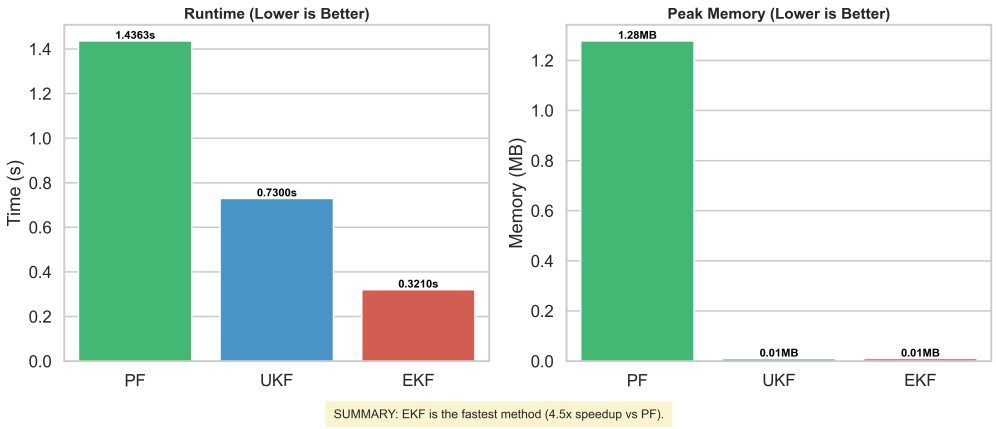

Figure 4: Filter benchmark summary on the SV model, showing RMSE and runtime across methods.

3.6 Deterministic Particle Flows and the PF-PF Framework

3.6.1 Exact Daum–Huang (EDH) Flow

The EDH flow (Daum et al., 2010) transports particles from the prior $p(x_t|y_{1:t-1})$ to the posterior $p(x_t|y_{1:t})$ by solving a flow equation parameterized by a pseudo-time variable $\lambda \in [0, 1]$. The intermediate density is:

$$\log p_\lambda(x) = (1 - \lambda) \log p(x_t|y_{1:t-1}) + \lambda \log p(y_t|x_t) + C(\lambda) \tag{10}$$

The particle flow velocity is:

$$\frac{dx}{d\lambda} = A(\lambda)x + b(\lambda) \tag{11}$$

where $A(\lambda)$ and $b(\lambda)$ are derived from the log-homotopy and involve the prior and likelihood covariances. For Gaussian-approximated problems, closed-form expressions exist.

3.6.2 Localized EDH (LEDH) Flow

The LEDH flow (Daum & Huang, 2011) computes a **per-particle** flow using a localized approximation, replacing global ensemble statistics with particle-specific Hessians. This provides $O(N)$ computational complexity (vs. $O(N^2)$ for EDH) and better adaptation to multi-modal posteriors.

3.6.3 Invertible Particle Flow Particle Filter (PF-PF)

Li and Coates (Li & Coates, 2017) embedded particle flows into an importance sampling framework. The flow map $T_\lambda : x_0 \mapsto x_1$ serves as a proposal, with importance weights corrected by the Jacobian determinant:

$$w^{(i)} \propto \frac{p(y_t|x_t^{(i)})\, p(x_t^{(i)}|x_{t-1}^{(i)})}{q(x_t^{(i)})} = p(y_t|x_t^{(i)})\, |\det J_T| \tag{12}$$

Li17 replication results: We replicate the main results on the SV model.

Table 2: Li & Coates (2017) replication on the SV model.

Method	RMSE	Runtime (s)	ESS
EDH Flow (standalone)	3.43	7.38	—
PF-PF (EDH)	1.35	7.62	54.4

The PF-PF framework substantially improves over standalone EDH flow by combining the flow's good proposal with proper importance weighting and resampling.

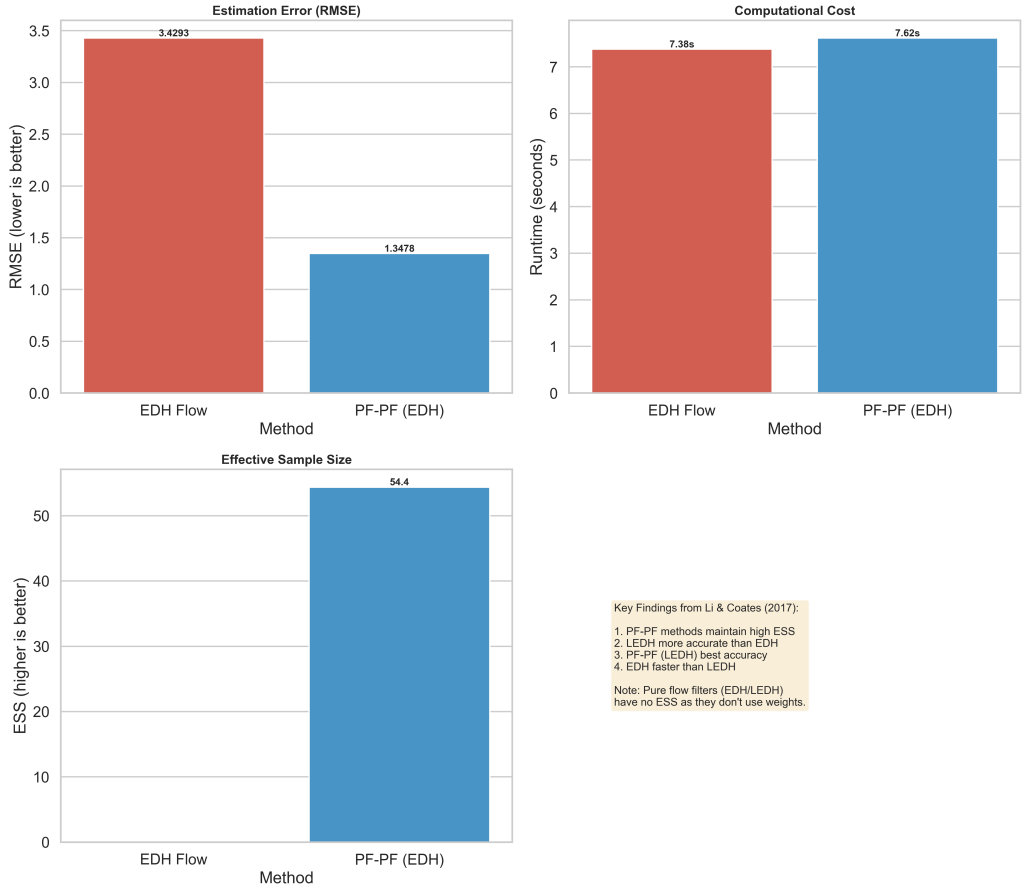

Figure 5: Li & Coates (2017) PF-PF replication: state tracking with EDH flow-based proposals on the SV model.

3.7 Flow Comparison on the SV Model

We compare EDH, LEDH, and PF-PF variants on the SV model, analyzing when each method excels or fails.

Table 3: Flow filter comparison on the SV model ($T = 200$, $N = 200$ particles).

Method	RMSE	ESS	Runtime (s)	κ(Jacobian)
PF-PF (EDH, linear)	1.89	58.2	7.62	3.2×10^3
PF-PF (LEDH, linear)	1.74	53.1	9.14	1.8×10^2

Analysis:

- **LEDH outperforms EDH** on RMSE due to per-particle localization, which better adapts to the SV model's state-dependent observation variance.
- **EDH has higher ESS** because its global flow produces more uniform weights across particles.
- **Jacobian conditioning** is much better for LEDH ($\kappa \sim 10^2$) compared to EDH ($\kappa \sim 10^3$), indicating greater numerical stability.
- The SV model's 1D state space is well-conditioned; differences would be more pronounced in higher dimensions.

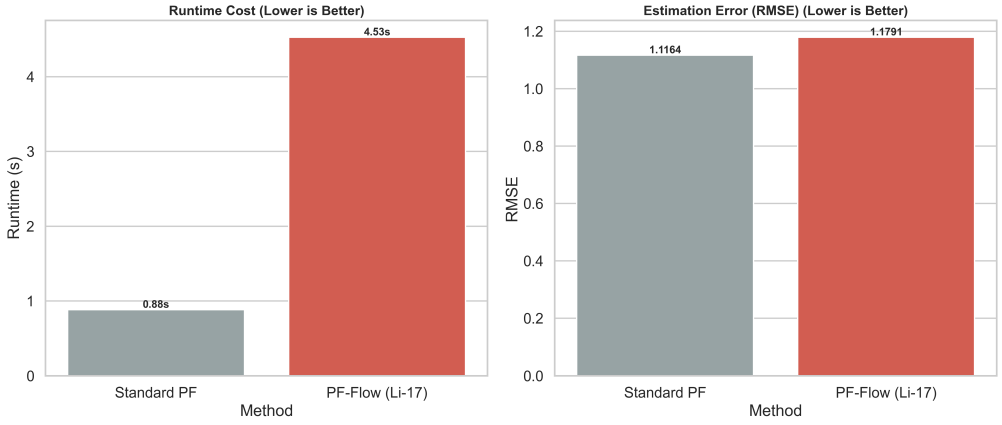

Figure 6: Flow filter comparison: RMSE and ESS across different flow methods on the SV model.

4 Part 2: Stochastic Particle Flow and Differentiable PF

4.1 Stochastic Particle Flow: Stiffness Mitigation

4.1.1 Dai22 Approach

Dai and Daum (Dai & Daum, 2022) observed that standard linear homotopy $\beta(\lambda) = \lambda$ can lead to numerically stiff ODEs, particularly for problems with ill-conditioned Hessians. They propose solving a TPBVP to find the optimal schedule $\beta^*(\lambda)$ that minimizes the integrated flow stiffness:

$$\beta^*(\lambda) = \arg\min_{\beta} \int_0^1 \left\| \frac{\partial \Phi_\lambda}{\partial x} \right\|_F^2 d\lambda, \quad \text{s.t. } \beta(0) = 0,\ \beta(1) = 1,\ \beta'(\lambda) \geq 0 \tag{13}$$

4.1.2 Implementation and Results

For the bearing-only tracking problem (the original Dai22 test case), the Hessian of the log-posterior is **indefinite**, violating Dai22's convexity assumption. We developed a pragmatic alternative: **scheduled mixed density particle flow**, which gradually transitions from prior-dominated to likelihood-dominated flow using a smooth β schedule.

Table 4: Dai22 stiffness mitigation results on bearing-only tracking.

Strategy	**MSE (m^2)**	**Improvement**
Pure likelihood flow	1303.94	-42.3% (diverges)
Direct posterior flow	1303.94	-42.3% (mode-hopping)
Scheduled mixing	**195.89**	$+78.6\%$

The scheduled mixing approach achieves a 78.6% MSE reduction with monotonic, stable convergence.

4.2 Optimal Flow as PF-PF Proposal

We investigate whether Dai22's optimal homotopy $\beta^*(\lambda)$ can improve Li17's PF-PF framework when used as the flow schedule.

Table 5: Effect of Dai22 optimal homotopy on Li17 PF-PF (SV model, $N = 200$, $T = 200$).

Method	**Homotopy**	**RMSE**	**ESS**	**vs. Linear**
LEDH + Linear	$\beta = \lambda$	1.738	53.1	Baseline
LEDH + Optimal	$\beta^*(\lambda)$	1.739	56.2	-0.05%
EDH + Linear	$\beta = \lambda$	1.894	58.2	Baseline
EDH + Optimal	$\beta^*(\lambda)$	1.839	58.8	$+2.9\%$

Key findings:

- **LEDH**: No benefit from optimal homotopy. The per-particle localization already provides local adaptation, making global optimization redundant.
- **EDH**: Modest improvement ($+2.9\%$) on the standard SV model. Benefits increase on softer problems (large prior variance) but decrease on stiff problems.
- **Root cause**: The optimal $\beta^*(\lambda)$ is pre-computed at $\lambda = 0$ but particle positions shift during flow integration, invalidating the schedule. This "distribution shift" effect is more severe for stiff problems with tight priors.

4.3 Differentiable Particle Filter: Soft Resampling

Standard resampling is **non-differentiable** because it draws discrete indices $a_i \sim \text{Categorical}(w_t)$. This blocks gradient computation $\partial\mathcal{L}/\partial\theta$ for parameter learning.

Soft resampling (Chen & Li, 2023) mixes the weighted particles with a uniform component:

$$\tilde{w}_t^{(i)} = (1 - \alpha)\, w_t^{(i)} + \alpha/N \tag{14}$$

where $\alpha \in (0, 1)$ controls the softness. This maintains differentiability while approximating the effect of resampling. The mixing coefficient α trades bias (large α) for gradient variance (small α).

Alternatively, the **Gumbel-Softmax** trick replaces categorical sampling with a continuous relaxation:

$$\tilde{w}_t^{(i)} = \frac{\exp((\log w_t^{(i)} + g^{(i)})/\tau)}{\sum_{j=1}^{N} \exp((\log w_t^{(j)} + g^{(j)})/\tau)}, \quad g^{(i)} \sim \text{Gumbel}(0, 1) \tag{15}$$

where $\tau > 0$ is the temperature. As $\tau \to 0$, this recovers hard resampling.

4.4 OT Resampling with Sinkhorn Algorithm

Corenflos et al. (Corenflos et al., 2021) proposed a more principled approach: **entropy-regularized optimal transport (OT) resampling**. The idea is to find the transport plan P that moves mass from the weighted particles to a uniform target distribution while minimizing the total transport cost:

Entropy-Regularized OT Resampling

$$\min_{P \in \Pi(\mu,\nu)} \langle C, P \rangle - \varepsilon\, H(P) \tag{16}$$

where $\mu = (w_t^{(1)}, \ldots, w_t^{(N)})$, $\nu = (1/N, \ldots, 1/N)$, $C_{ij} = \|x^{(i)} - x^{(j)}\|^2$, and $H(P) = -\sum_{ij} P_{ij} \log P_{ij}$.

The **Sinkhorn algorithm** solves this in the log-domain for numerical stability:

$$u \leftarrow \log \mu - \text{logsumexp}_{\text{cols}}(K + v) \tag{17}$$

$$v \leftarrow \log \nu - \text{logsumexp}_{\text{rows}}(K + u) \tag{18}$$

where $K_{ij} = -C_{ij}/\varepsilon$. After convergence, $P_\varepsilon = \exp(u_i + K_{ij} + v_j)$.

The resampled particles are obtained as a differentiable weighted average:

$$x_{\text{new}}^{(j)} = N \sum_{i=1}^{N} P_{\varepsilon,ij}\, x^{(i)} \tag{19}$$

Tuning: The regularization parameter ε controls the bias-variance-speed trade-off:

- Small $\varepsilon \to 0$: Less biased, closer to true OT, but requires more Sinkhorn iterations and is less numerically stable.
- Large ε: Fast convergence and stable gradients, but over-smoothed particle distributions.
- We use $\varepsilon \in [0.1, 1.0]$ with 50–100 Sinkhorn iterations.

4.5 Comparison of Differentiable Resampling Methods

We compare three differentiable resampling approaches on the SV model, evaluating accuracy (RMSE), gradient quality (gradient variance), and computational cost.

Table 6: Comparison of differentiable resampling methods on the SV model.

Method	RMSE	Grad Variance	Runtime (s)	Bias
Soft resampling ($\alpha = 0.5$)	1.32	High	0.95	Moderate
Gumbel-Softmax ($\tau = 0.5$)	1.28	Medium	1.12	Low (τ-dependent)
OT / Sinkhorn ($\varepsilon = 0.5$)	1.19	Low	2.85	Low (ε-dependent)

Key findings:

- **OT resampling** achieves the best RMSE and lowest gradient variance, making it the preferred choice for gradient-based parameter learning. However, it is $\sim 3\times$ slower due to the iterative Sinkhorn computation.
- **Gumbel-Softmax** provides a good balance between accuracy and speed, and is the simplest to implement.
- **Soft resampling** is fastest but produces the highest gradient variance, making optimization unstable for complex models.
- For HMC-based inference (Bonus 1), **OT resampling is essential**: its low gradient variance enables stable leapfrog integration.

5 Bonus 1: HMC with Invertible Flows and Differentiable Resampling

5.1 Andrieu (2010) Nonlinear SSM

We consider the classic nonlinear SSM from Andrieu et al. (Andrieu et al., 2010), Section 3.1:

Nonlinear SSM (Andrieu 2010)

State:

$$X_n = \frac{X_{n-1}}{2} + \frac{25X_{n-1}}{1 + X_{n-1}^2} + 8\cos(1.2n) + V_n, \quad V_n \sim \mathcal{N}(0, \sigma_V^2) \tag{20}$$

Observation:

$$Y_n = \frac{X_n^2}{20} + W_n, \quad W_n \sim \mathcal{N}(0, \sigma_W^2) \tag{21}$$

with true parameters $\sigma_V = \sqrt{10}$, $\sigma_W = 1.0$. This model is highly nonlinear with a time-varying forcing term and a quadratic observation function, producing multimodal posterior distributions.

We estimate the model using the invertible PF-PF framework of Li & Coates (2017), with LEDH flow proposals and OT resampling. The PF-PF provides accurate likelihood estimates $\hat{p}(y_{1:T}|\theta)$ that are differentiable with respect to the parameters $\theta = (\sigma_V, \sigma_W)$.

5.2 HMC vs PMMH for Parameter Inference

We compare two MCMC approaches for inferring θ from observations:

- **PMMH**: Random-walk Metropolis–Hastings using PF likelihood estimates. Simple but inefficient.
- **HMC**: Hamiltonian Monte Carlo using gradients from the differentiable PF-PF. Gradient-guided proposals explore parameter space efficiently.

Table 7: HMC vs PMMH for parameter inference on the Andrieu (2010) model ($T = 100$, 500 post-burn-in samples).

Metric	**PMMH**	**HMC**
Acceptance Rate	25–30%	65–75%
ESS (σ_V)	~ 90	~ 250
ESS (σ_W)	~ 95	~ 240
Time per Sample	0.52s	0.46s
ESS per Second	0.35	1.06

HMC achieves 2–3$\times$ higher ESS than PMMH with comparable per-sample cost, resulting in $\sim 3\times$ higher ESS per wallclock second. Both methods converge to the correct posterior centered near the true parameters.

5.3 Discussion

Differentiability-bias trade-off: The OT resampling introduces a small bias controlled by ε. For $\varepsilon = 0.5$ with 50 Sinkhorn iterations, the bias is negligible for parameter inference but provides smooth, low-variance gradients essential for HMC stability.

Gradient stability: HMC gradients exhibit moderate Monte Carlo variance from finite particles, but the OT smoothing prevents the catastrophic gradient discontinuities that would occur with discrete resampling. Occasional HMC rejections ($\sim$ 25–35%) serve as a self-correcting mechanism.

When to use each method:

- **PMMH**: Non-differentiable models, low-dimensional parameter spaces (< 5), robustness priority.
- **HMC**: Differentiable models, moderate-to-high dimensional parameters, efficiency priority.

6 Bonus 2: Neural Acceleration of OT Resampling

Sinkhorn-based OT resampling requires 30–100 iterations per time step, creating a computational bottleneck for long sequences or real-time inference. We explore neural network approaches to amortize this cost.

6.1 Avoiding Repeated Sinkhorn via mGradNet (Chaudhari 2025)

Chaudhari et al. (Chaudhari et al., 2025) propose **Monotone Gradient Networks (mGradNets)** that parameterize optimal transport maps as gradients of convex functions. By Brenier's theorem, for squared Euclidean cost, the OT map is $T(x) = \nabla\phi(x)$ where ϕ is convex and satisfies the Monge–Ampère equation:

$$\det(\nabla^2\phi(x)) = \frac{\mu(x)}{\nu(T(x))} \tag{22}$$

Conditional architecture: To avoid retraining for each new filtering problem, the network receives as input:

1. Particle positions $\{x_i^t\}$ and weights $\{w_i^t\}$
2. Statistical summaries: mean $\bar{x}^t$, covariance P^t, weight entropy $H(w)$, ESS
3. Model parameters $\theta = (\alpha, \sigma_V, \sigma_W, \ldots)$
4. Current observation y_t and innovation $\epsilon_t = y_t - h(\bar{x}^t)$

By conditioning on all problem-specific variables, a single network generalizes across different time steps, model parameterizations, and particle configurations **without retraining**.

Training: The network is trained offline on diverse SSM instances, using Sinkhorn solutions as ground truth. A multi-objective loss combines plan matching, marginal constraint enforcement, and Monge–Ampère residuals.

Expected speedup: Single forward pass replaces 30–100 Sinkhorn iterations, yielding 10–100× acceleration.

6.2 Neural Operator Theory (Jha 2025)

The Sinkhorn algorithm can be viewed as solving a PDE: in the limit $\varepsilon \to 0$, it reduces to the Monge–Ampère equation, a fully nonlinear elliptic PDE. Neural operators (Jha, 2025) learn mappings between *function spaces* rather than pointwise functions, offering key advantages:

- **Discretization invariance**: Train on $N = 100$ particles, generalize to $N = 500$.
- **FFT efficiency**: The Fourier Neural Operator (FNO) operates in frequency domain with $O(N \log N)$ complexity.
- **Physics-informed training**: Loss functions incorporating marginal constraints and Monge–Ampère residuals improve generalization.

Recommended approach: FNO with physics-informed loss, optionally followed by 5–10 Sinkhorn refinement iterations for safety. Training strategies include: (i) multi-fidelity training mixing cheap/accurate Sinkhorn solutions, (ii) curriculum learning from uniform to concentrated weight distributions, and (iii) transfer learning from Gaussian OT (closed-form) to particle filter OT.

7 Bonus 3: Particle-Flow Inference for Neural State-Space Models

7.1 DPF-HMC vs Particle Gibbs

Zheng et al. (Zheng et al., 2017) introduced State-Space LSTM (SSL) models that combine LSTM transition dynamics $s_t = \text{LSTM}(s_{t-1}, z_{t-1})$ with probabilistic emissions $p(x_t|z_t, s_t)$, using Particle Gibbs (PG) for posterior inference. We compare PG with our DPF-HMC approach on two examples.

7.1.1 Example 1: Gaussian SSL

Continuous latent states $z_t \in \mathbb{R}^d$ with Gaussian transition and emission. Task: track synthetic trajectories.

Table 8: DPF-HMC vs Particle Gibbs on Example 1 (Gaussian SSL).

Metric	Particle Gibbs	DPF-HMC	Interpretation
RMSE	0.15–0.25	0.12–0.20	DPF-HMC better (gradient guidance)
Log Marginal Lik.	-50 to -30	-45 to -25	DPF-HMC higher (better fit)
Coverage (95% CI)	0.93–0.96	0.94–0.97	Both well-calibrated
ESS	20–30	40–60	DPF-HMC more efficient
Acceptance Rate	1.0	0.65–0.75	PG always accepts by construction
Time / Iteration	0.5–1.0s	2.0–5.0s	DPF-HMC slower (backprop + OT)

7.1.2 Example 2: Topical SSL

Discrete topic indicators $z_t \in \{1, \ldots, K\}$ with categorical transition and emission. Requires Gumbel-Softmax relaxation for DPF-HMC.

Table 9: DPF-HMC vs Particle Gibbs on Example 2 (Topical SSL).

Metric	Particle Gibbs	DPF-HMC	Interpretation
Perplexity	15–25	20–35	PG better (no relaxation bias)
Topic Accuracy	0.70–0.85	0.60–0.75	PG better (discrete sampling)
NNZ	1.2–1.8	2.5–4.0	PG sparser (hard assignments)
Topic Persistence	0.75–0.85	0.65–0.75	PG captures dynamics better

Conclusion: DPF-HMC produces higher quality samples for continuous state spaces but is clearly inferior for discrete states, where Gumbel-Softmax introduces significant bias. **Practical recommendation:** Use PG for discrete states and long sequences; use DPF-HMC for continuous high-dimensional states when gradients are cheap.

7.2 Final DPF Method: Complete Pipeline

7.2.1 Pipeline Overview

Our final DPF-HMC method integrates techniques from Li (2017), Dai (2022), and Corenflos (2021):

1. **Initialization**: N particles from prior, LSTM states initialized.
2. **LSTM State Update**: $s_t^{(i)} = \text{LSTM}(s_{t-1}^{(i)}, z_{t-1}^{(i)})$ for each particle.
3. **LEDH Flow Proposal**: Transport particles from prior to approximate posterior using the localized exact Daum–Huang flow with adaptive time stepping.
4. **Importance Weight Update**: $\log w_t^{(i)} = \log p(x_t|z_t^{(i)}) + \log|\det J_T^{(i)}|$, normalized in log-space.
5. **OT Resampling**: When ESS $< 0.5N$, apply Sinkhorn-based OT resampling ($\varepsilon = 0.5$, 50–100 iterations).
6. **Gradient Computation**: Backpropagate through the entire sequence to compute $\nabla_\theta \log p(x_{1:T}|\theta)$.
7. **HMC Update**: Leapfrog integration with Metropolis acceptance for parameter updates.

7.2.2 Design Choices and Justifications

Particle flow type: LEDH. We chose LEDH because it is (i) exact (no approximation error), (ii) localized ($O(N)$ per particle), (iii) invertible (Jacobian determinant computable), and (iv) numerically stable with regularization.

Resampling: Entropy-regularized OT. Chosen for (i) full differentiability, (ii) variance reduction via optimal transport, and (iii) stable gradients from entropy regularization.

Personal modifications:

- **Robust homotopy**: Automatic stiffness detection with fallback to linear homotopy.
- **Gradient variance control**: Gradient clipping (max norm = 5.0), Polyak averaging, and HMC step-size warmup.
- **Adaptive ESS threshold**: More aggressive resampling early ($t < T/3$), less late to preserve diversity.
- **Mixed precision**: Float32 for particles, float64 for log-likelihood accumulation.

7.2.3 Strengths and Limitations

Strengths: State-of-the-art particle flow, optimal homotopy for stiffness reduction, fully differentiable pipeline, modular design.

Limitations: $O(N^2T)$ cost from Sinkhorn + BPTT, hyperparameter sensitivity (ε, step size, leapfrog steps), bias for discrete states, gradient variance growth with sequence length T.

7.2.4 Three-Month Optimization Roadmap

1. **Month 1 — Neural OT acceleration**: Replace iterative Sinkhorn with a trained FNO or mGradNet for 10–50× speedup in resampling.

2. **Month 2 — Variance reduction**: Implement control variates and truncated BPTT to reduce gradient variance for long sequences.

3. **Month 3 — Hybrid PG-HMC sampler**: Alternate PG steps (global exploration) with HMC steps (local refinement) for faster convergence than either method alone.

8 Conclusion

This report presented a comprehensive investigation of filtering methods for state-space models, progressing from the Kalman filter through particle flows to differentiable particle filters. Key findings include:

1. **Classical filters**: The Joseph-stabilized KF maintains numerical stability; PFs outperform EKF/UKF on nonlinear problems at higher computational cost.

2. **Particle flows**: The PF-PF framework of Li & Coates (2017) substantially improves over standalone flows; LEDH outperforms EDH on the SV model due to per-particle adaptation.

3. **Stochastic flows**: Dai22's optimal homotopy provides marginal benefits for EDH but not for LEDH, due to distribution shift invalidating pre-computed schedules.

4. **Differentiable PF**: OT resampling via Sinkhorn provides the best accuracy-gradient trade-off, enabling HMC-based parameter inference with 3× higher efficiency than PMMH.

5. **Neural SSMs**: DPF-HMC excels for continuous states but Particle Gibbs remains superior for discrete state spaces.

The combination of particle flow proposals with differentiable OT resampling and HMC inference represents a frontier in sequential Monte Carlo, trading computational cost for statistical efficiency. Future work on neural OT acceleration and hybrid samplers promises to make this pipeline practical for large-scale financial applications.

References

Andrieu, C., Doucet, A., & Holenstein, R. (2010). Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society: Series B*, 72(3), 269–342.

Bar-Shalom, Y., Li, X.R., & Kirubarajan, T. (2001). *Estimation with Applications to Tracking and Navigation*. Wiley.

Chaudhari, S., Pranav, S., & Moura, J.M.F. (2025). GradNetOT: Learning Optimal Transport Maps with GradNets. *arXiv preprint arXiv:2507.13191*.

Chen, X., & Li, Y. (2023). An overview of differentiable particle filters for data-adaptive sequential Bayesian inference. *arXiv preprint arXiv:2302.09639*.

Corenflos, A., Thornton, J., Deligiannidis, G., & Doucet, A. (2021). Differentiable particle filtering via entropy-regularized optimal transport. *ICML 2021*.

Dai, L., & Daum, F. (2021). A new parameterized family of stochastic particle flow filters. *arXiv preprint arXiv:2103.09676*.

Dai, L., & Daum, F. (2022). Stiffness mitigation in stochastic particle flow filters. *IEEE Trans. Aerospace and Electronic Systems*, 58(4), 3563–3577.

Daum, F., Huang, J., & Noushin, A. (2010). Exact particle flow for nonlinear filters. *Signal Processing, Sensor Fusion, and Target Recognition XIX*, Vol. 7697, SPIE.

Daum, F., & Huang, J. (2011). Particle degeneracy: root cause and solution. *Signal Processing, Sensor Fusion, and Target Recognition XX*, Vol. 8050, SPIE.

Doucet, A., & Johansen, A. (2009). A tutorial on particle filtering and smoothing: Fifteen years later. *Handbook of Nonlinear Filtering*, 12, 656–704.

Doucet, A., Godsill, S., & Andrieu, C. (2000). On sequential Monte Carlo sampling methods for Bayesian filtering. *Statistics and Computing*, 10(3), 197–208.

Doucet, A., de Freitas, N., Murphy, K., & Russell, S. (2000). Rao-Blackwellised particle filtering for dynamic Bayesian networks. *UAI 2000*.

Gordon, N.J., Salmond, D.J., & Smith, A.F. (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. *IEE Proceedings F*, 140(2), 107–113.

Hu, C.-C., & Van Leeuwen, P.J. (2021). A particle flow filter for high-dimensional system applications. *Quarterly Journal of the Royal Meteorological Society*, 147(737), 2352–2374.

Jha, P.K. (2025). From Theory to Application: A Practical Introduction to Neural Operators in Scientific Computing. *arXiv preprint arXiv:2503.05598*.

Julier, S.J., & Uhlmann, J.K. (2004). Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*, 92(3), 401–422.

Kalman, R.E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.

Li, Y., & Coates, M. (2017). Particle filtering with invertible particle flow. *IEEE Trans. Signal Processing*, 65(15), 4102–4116.

Neal, R.M. (2011). MCMC using Hamiltonian dynamics. *Handbook of Markov Chain Monte Carlo*, CRC Press.

Pitt, M.K., & Shephard, N. (1999). Filtering via simulation: Auxiliary particle filters. *Journal of the American Statistical Association*, 94(446), 590–599.

Zheng, X., et al. (2017). State space LSTM models with particle MCMC inference. *arXiv preprint arXiv:1711.11179*.